

Video Transcript

Video 8: Spring Boot: A Web Developing Application for Java (09:54)

Programming languages are pretty compact, so when it comes to creating an application, there's more that's needed. And because of it, communities create application frameworks and the Java domain. One of the most popular, one of the most widely used is Spring Boot. As you can see here, Spring is the application framework, and Spring Boot goes even farther. In fact, there's a website that they have created that makes it very easy for you to create an application. And so that's what we're going to do. We're going to start off by navigating to that website, creating that application, downloading those files onto our host machine.

We will then create a container, that container will have everything that you need to run these applications. We will add to it an editor and also expose a port, as you can see. We're then going to upload that code that we have downloaded onto our host machine, and then we're going to make some configurations to do our 'Hello World', and then we're going to go ahead and run it. So, let's get started by navigating to that website and creating our application. And as you can see here on my desktop, I have opened up a browser. Next to it I have a directory file system. This is pointing to a folder on my desktop. You can see desktop sample on the path and then below it I have the same, except that I am at a terminal. And so, you can see they are the same path as well.

So, I'm going to go ahead and navigate to that address. And you can see here that I have start.spring.io, that is the address that I'm navigating to. And we're going to go ahead and select or leave the default, as you can see there, it is using the project Maven. It is using the language of Java, the version of Spring Boot that we're using, the project metadata. We're going to leave all of that as default, and we're simply going to add one dependency to be able to create a web server, and we're going to create within it our 'Hello world'. So, that's all we're going to do. We're then going to go ahead and generate this application. And as you can see, that file, that zip has been downloaded onto my machine, the host machine. Now, I'm going to go ahead and drag and drop that zip onto my sample directory, as you can see there. And so I have them here at my command line. I can list the files as well.

And so having done so, we're going to go ahead and move on to our next step, which is to create the container. Remember, we're going to expose a port and add an editor. And as you can see, here is the command line to be able to do that. I'm going to go ahead and run that. I'm going to go ahead and that's it. That's going to open up to a prompt. And you can see there that I'm using Maven, which is required for this environment. This project is from the Apache Software Foundation if you want to learn more about it. You can see there that I am exposing the port 8080 and that I'm naming that container javamaven.

Now, below it are just the lines to be able to add in editor once I create the container. I've opened my window full size and now I'm going to go ahead and paste that command onto it. And as you can see there, that moves pretty quickly through the installation and now we have our command prompt. If I list my files, you can see there that I am at the root, I have our command prompt that I've just created. I'm now going to go ahead and install the editor.

And now that, that is installed, we have a container configured with the right development environment and an editor for us to use. So, our next step is to upload the code, and we will do so from the host. As you can see here, once again, I am at the desktop inside of the sample folder and we're going to be copying the files that are inside of demo and we're going to be copying those files to the home directory. I've chosen that one because it's empty. And so, now I'm going to go ahead and enter the command. And as you can see there that is Docker copy demo pointing to the files inside of the demo directory here locally on the host machine. And then I'm going to copy that to the container. The container is called javamaven and inside of it to the home directory.

So, once I hit that and enter it, then if we list our files here inside of the container, we can see the same files that we have on the host. Once we have moved our code into the container, then the next step that we want to do is that we want to modify slightly that application that we just uploaded. And we're going to do so by adding this code. This is a template application, as you might imagine, the one that we just downloaded. It's just a trimmed template to get you going on web applications, and all we're going to do is that we're going to replace the code that we will see in a file in a moment with the following.

This one is simply doing the 'Hello World', and as you can see there at the bottom, it takes two parameters and if no parameter is given, it simply returns 'Hello World'. Otherwise, it returns Hello and the name that was provided. So, let's go ahead and move to the container and inside of

it we're going to navigate to the source. We start off by the source directory, the path that we're going to navigate through is painful and this has to do with how this platform does directories, how Java does name spaces. And as you can see there, I am going deep, deep into this code. I'm still going through it. Let's see, have I gotten to it? Yes. You can see there that I am at the demo application. And now I'm going to run my editor and edit that code. And as you can see there, I have the default code and we're going to replace that with what we saw on the slide. So, I'm going to go ahead and remove this code and replace it as so. Now, I'm going to go ahead and save this file, confirm my changes. And as we can see there, we have the code with that method there at the end that takes those parameters.

So, our last step now is to try it. As you can see there, we're going to access that from our host machine. We'll be opening up a browser that will hit that container in the port that it opened, the 8080 port that was opened at the very beginning and that is the same port that is running inside of the container within that application. But before we do that, we need to run the application and you can see here what the command is. Let's paste the command. And as that installs, I'm going to go ahead and open up a browser.

So, it looks like I have an error. Yes. And the reason for that is that I am not at the root of the application. So, let's go ahead and move into Home. It's looking for the dependencies, It's looking for the file that defines the application. And you can see there that pom is the one that does that, If we were to look through it, and I will leave that out to you, you could see the definition of the application. So, because it couldn't find it, it got stuck. So, I'm going to go ahead and run it now and the application is now up and running and listening.

I'm going to go ahead and navigate to the path. So, that would be localhost and we're going to go ahead and enter 'Hello'. And once we do, once we do, you can see there that we get 'Hello World!' There were two options or two parameters provided there, as you saw in that method inside of the application. If you provided no parameter accompanying just the path that we defined, then you got the default 'Hello World'. However, if we passed in a value, and in this case the value that we're going to pass in is going to be Peter, then we should get 'Peter'. And yes, we do.

Conceptually, we did quite a bit. So, let's review. We started out by using some scaffolding, the one that is provided for us by that website, start.spring.io, and we created there our application. We downloaded that onto our host machine. Then we created a container. We copied that code

into the container. Then we added an editor, we edited the code, we created our 'Hello World', and then we ran that application in the container and accessed that externally. Now,, there's a lot going on here from an architecture standpoint. There are several packages being used, there are ports being mapped, there's a container being used on top of a host machine. So, go through this application in detail, make sure you understand all of those abstractions and what the architecture is. Definitely make that code run for you. Make some methods, make some changes, and make sure that what you think is taking place is in fact happening. So, now it's your turn.

